1. Aufgabe: Versionierung/Versionsverwaltung (10 Punkte)

- Was versteht man im Zusammenhang mit Versionsverwaltung unter Merge?
  - Ein Merge (Dateiverschmelzung) ist das Integrieren von bereits im Repository vorhandenen Änderungen in die eigene, lokale Datei. Dieser Schritt ist beim Einchecken notwendig, wenn eine Datei von mehreren Entwicklern gleichzeitig ausgecheckt und in der Zwischenzeit bereits von einem anderen Entwickler aktualisiert wurde. Die Zusammenführung dieser beiden Dateiversionen kann dabei entweder völlig konfliktfrei erfolgen oder zu einem Konflikt führen.

- Was versteht man im Zusammenhang mit Versionsverwaltung unter Einchecken mit einem Konflikt?
  - Ein Konflikt beim Einchecken entsteht, wenn Entwickler dieselbe Datei parallel bearbeiten und einer seine Version später einreichen möchte. Zur Lösung muss dieser Entwickler zunächst die bereits im Repository gespeicherten Änderungen in seine lokale Datei integrieren (Merge). Erst nach dieser erfolgreichen Zusammenführung beider Bearbeitungsstände kann die finale, kombinierte Version eingecheckt werden.

2. Aufgabe: Design Patterns (45 Punkte)

a)

```
public class RestaurantsModel {
    private Restaurant restaurant;

    public RestaurantsModel() {
        this.restaurant = new Restaurant(
                                "Test Restaurant", 15, 2020, 2023);
    }

    public Restaurant getRestaurant() {
        return restaurant;
    }

    public void setRestaurant(Restaurant restaurant) {
        this.restaurant = restaurant;
    }
}


public class RestaurantsControl {
    private RestaurantsView restaurantsView;
    private RestaurantsModel restarantsModel;

    public RestaurantsControl(Stage primaryStage){
        this.restarantsModel = new RestaurantsModel();
        this.restaurantsView = new RestaurantsView(
                this, primaryStage, restarantsModel);
    }
}


public class RestaurantsView {
    private RestaurantsModel restaurantsModel;
    private RestaurantsControl restaurantsControl;

    ...


    public RestaurantsView(
            RestaurantsControl restaurantsControl,
            Stage primaryStage,
            RestaurantsModel restaurantsModel
    ){
        this.restaurantsControl = restaurantsControl;
        this.restaurantsModel = restaurantsModel;

        ...
    }
```

b)

```
public class RestaurantsModel {
    private Restaurant restaurant;
    private static RestaurantsModel theInstance = null;

    private RestaurantsModel() {
        this.restaurant = new Restaurant(
                                "Test Restaurant", 15, 2020, 2023);
    }

    public static RestaurantsModel getInstance() {
        if(theInstance == null) {
            theInstance = new RestaurantsModel();
        }
        return theInstance;
    }

    public Restaurant getRestaurant() {
        return restaurant;
    }

    public void setRestaurant(Restaurant restaurant) {
        this.restaurant = restaurant;
    }
}
```

```
public class RestaurantsControl {
    private RestaurantsView restaurantsView;
    private RestaurantsModel restarantsModel;

    public RestaurantsControl(Stage primaryStage){
        this.restarantsModel = RestaurantsModel.getInstance();
        // this.restarantsModel = new RestaurantsModel();
        this.restaurantsView = new RestaurantsView(
                this, primaryStage, restarantsModel);
    }
}
```

c)

```
public class RestaurantsModel {
        ...
        public void leseRestaurantAusTxt() throws IOException {
        BufferedReader ein =
              new BufferedReader(new FileReader("Text.txt"));
        this.restaurant = new Restaurant(
                ein.readLine(),
                Integer.parseInt(ein.readLine()),
                Integer.parseInt(ein.readLine()),
                Integer.parseInt(ein.readLine())
                );
        ein.close();
        }
        ...
}

public class RestaurantsControl {
        ...
        public void leseRestaurantAusTxt(){
                try {
                        this.restarantsModel.leseRestaurantAusTxt();
                } catch (IOException e) {
                        this.restaurantsView.zeigeFehlermeldungAn(
                                "Fehlermeldung beim Lesen von Text.txt");
                }
        }
        ...
}

public class RestaurantsView {
        ...
      private void initListener(){
          btnRestaurants.setOnAction(new EventHandler<ActionEvent>() {
              @Override
              public void handle(ActionEvent event) {
                  restaurantsControl.leseRestaurantAusTxt();
                  setTxtRestaurants(restaurantsModel.getRestaurant()
                                                  .gibAttributeZurueck());
              }
          });
      }

      void setTxtRestaurants(String txt){
          this.txtRestaurants.setText(txt);
      }
      ...
}
```

3. Aufgabe: Vertiefung Java (35 Punkte)

a)

```
public class Anwendersystem {
    public static void main(String[] args) {
        ...
        Restaurant restaurantHa =
         new Restaurant("Hamburg Nord", 23, 2012, 2015);

        RestaurantRestaurantDrivein restaurantBr =
                       new RestaurantRestaurantDrivein(
                              "Bremen West", 27, 1908, 2016, true);

        ArrayList<Restaurant> restaurants = new ArrayList<>();
        restaurants.add(restaurantHa);
        restaurants.add(restaurantBr);

        for(Restaurant restaurant: restaurants){
               System.out.println(restaurant.gibAttributeZurueck());
        }
        ...
    }
}
```

b)

```
public class RestaurantUtil <T extends Restaurant>{
    private T restaurantBasis;

    public RestaurantUtil(T restaurantBasis){
        this.restaurantBasis = restaurantBasis;
    }

    public void gibRestaurantBasisAus(){
        System.out.println(
               "Attribute: " + restaurantBasis.gibAttributeZurueck());
    }
}


public class Anwendersystem {
    public static void main(String[] args) {
        ...
        RestaurantUtil<Restaurant> restaurantUtilHa =
               new RestaurantUtil<>(restaurantHa);
        RestaurantUtil<RestaurantRestaurantDrivein> restaurantUtilBr =
               new RestaurantUtil<>(restaurantBr);
        restaurantUtilHa.gibRestaurantBasisAus();
        restaurantUtilBr.gibRestaurantBasisAus();
    }
}
```

c)

```
public class Anwendersystem {
    public static void main(String[] args) {
        Supplier<String> stringSupplier =
                () -> "Hat das Restaurant " + restaurantBr.getName()
                       + " einen Drivein? ";
        BooleanSupplier booleanSupplier =
                () -> restaurantBr.isMitRestaurantDrivein();
        System.out.println(stringSupplier.get() +
                booleanSupplier.getAsBoolean());
    }
}
```

4. Aufgabe: JUnit und Tests (30 Punkte)

```
public class RestaurantTest {
    private Restaurant restaurant;

    @BeforeEach
    public void setUp() throws Exception{
        restaurant = new Restaurant("Test Restaurant", 30, 2020, 2023);
    }

    @AfterEach
    public void tearDown() throws Exception{
        restaurant = null;
    }

    @Test
    public void test() throws Exception{
        int jahrLetzteRenovierung = restaurant.getEroeffnungsjahr() - 5;

        Throwable throwable1 = assertThrows(IllegalArgumentException.class,
            () -> this.restaurant
                        .setJahrLetzteRenovierung(jahrLetzteRenovierung));
        assertEquals(
                "Die letzte Renovierung darf nicht später als das
                       Eröffnungsjahr sein",
                throwable1.getMessage()
        );

        Throwable throwable2 = assertThrows(IllegalArgumentException.class,
                () -> this.restaurant = new Restaurant(
                                "Test Restaurant", 30, 2020, 2018));
        assertTrue(
                () -> throwable2.getMessage().equals("Die letzte
               Renovierung darf nicht später als das Eröffnungsjahr sein"),
                () -> "Die Fehlermeldung ist ungültig und nicht wie
                       erwartet"
        );
    }
}
```